

KRISHKUMAR KALYA

✉ krishkalya31012005@gmail.com | 📍 Pune, Maharashtra

in linkedin.com/in/krishkumar-kalya | 🌐 github.com/Krish3101 | 🌐 krish3101.github.io

EDUCATION

MIT ADT University

Pune, India

Bachelor of Technology in Computer Science and Engineering / **CGPA: 8.68**

Expected May 2027

Relevant Coursework: Data Structures & Algorithms, Database Management Systems, Object-Oriented Programming, Operating Systems, Computer Networks, Software Engineering

TECHNICAL SKILLS

Languages: Python, Java, SQL, JavaScript, TypeScript

Databases: PostgreSQL, PostGIS, SQLite, SQLAlchemy

Frameworks & Libraries: Spring Boot, FastAPI, Flask, Express, Spring Data JPA, GeoPandas

Tools & Platforms: Git, GitHub, Docker, Postman, Linux, Maven

EXPERIENCE

AI-ML Virtual Internship

April – June 2025

AICTE – EduSkills, supported by Google for Developers (India Edu Program)

Remote

- Completed the 10-week AICTE–EduSkills AI-ML virtual internship, supported by Google for Developers, for fifth-semester internship credit.
- Covered TensorFlow, image classification, object detection and product image search through Google Developers ML pathways, earning a skill badge for each.

PROJECTS

disputeledger | 🌐 GitHub | *TypeScript, Express, better-sqlite3, Zod*

- Built a dispute tracking system for supply chain partners: disputes are raised with evidence attached, arbiters resolve them, and every action is recorded on an append-only event log.
- Made the log tamper-evident by chaining events with SHA-256 — each event stores the previous event’s hash, so verification recomputes the chain and a row edited directly in the database breaks the link instead of passing unnoticed.
- Kept the dispute rules in a pure domain module with no I/O so they are testable on their own; authentication uses script-hashed passwords and server-side session tokens. Tested with Vitest and supertest.

geoagent | 🌐 GitHub | *pydantic-ai, FastAPI, GeoPandas, OSMnx, SQLite*

- Built a single async FastAPI service that maps Python geospatial functions into LLM tools through Pydantic-AI, so every model-issued call is validated against a typed contract instead of a hand-rolled planner loop.
- Implemented the spatial toolset over OpenStreetMap boundaries and Planetary Computer Sentinel-2 imagery, returning GeoJSON, GeoPackage and GeoTIFF files, with geocoding throttled to one request per second to respect the Nominatim usage policy.
- Constrained the model to tool selection only: coordinates and output paths stay in the run context and never appear in model output, so every geometry calculation is done by Python. Task progress streams to the browser over server-sent events.

trafficengine | 🌐 GitHub | *Java 21, Spring Boot 3.4, Spring Data JPA, H2*

- Built a rule engine in Java and Spring Boot that evaluates speed events against configurable thresholds and calculates tiered fines, exempting emergency vehicles before any penalty applies.
- Kept the rule engine as plain Java with no Spring and no database in it, so the rules are unit-testable without starting either; speed limits and fine tiers are configured in `application.yml` rather than compiled into the code.
- Tested the engine with JUnit 5 unit tests and MockMvc integration tests, and validated request payloads with Jakarta Bean Validation.

CERTIFICATIONS

Java Programming: Solving Problems with Software – Duke University, Coursera

Data Structures – University of California San Diego, Coursera

Advanced Java – LearnKartS, Coursera

Neural Networks and Deep Learning – DeepLearning.AI, Coursera